

Travail de session — Architecture sécurisée d'une plateforme pédagogique conteneurisée sur VM as Code

Informations générales

Pondération : 40 points, soit 40 % de la note finale.

Taille des équipes : 3 à 5 étudiants.

Format de réalisation : travail d'équipe.

Environnement cible : une machine virtuelle Linux initialisée par cloud-init et hébergeant une plateforme conteneurisée déployée avec Docker Compose.

Objectif pédagogique : démontrer des compétences de base en architecture de solutions conteneurisées, en sécurité des conteneurs, en défense en profondeur, en gestion des identités, en segmentation réseau, en observabilité et en automatisation de l'infrastructure.

Ce travail de session vise à vous placer dans une situation réaliste de conception et de déploiement d'une plateforme numérique auto-hébergée, tout en vous imposant une complexité raisonnable pour un premier projet de sécurité des conteneurs. Vous devrez livrer une architecture fonctionnelle, explicable, reproductible et suffisamment sécurisée pour illustrer les bonnes pratiques vues au cours, notamment le durcissement des images, le durcissement à l'exécution, la réduction des privilèges, la segmentation des réseaux, le durcissement de l'hôte, l'observabilité et l'intégration de contrôles de sécurité en amont du déploiement (shift left).[cite:10][cite:14][cite:51]

Mandat

Votre équipe doit concevoir, automatiser, déployer et documenter une plateforme pédagogique conteneurisée composée de plusieurs services open source intégrés entre eux. La plateforme doit être déployée sur une VM unique provisionnée en mode VM as Code à l'aide de cloud-init, puis exploitée au moyen de conteneurs orchestrés avec Docker Compose.[cite:10][cite:56]

L'architecture devra intégrer les composants suivants :

- **Traefik** comme reverse proxy d'entrée et contrôleur d'exposition web.
- **OpenAppSec** de Check Point comme couche WAF ou protection applicative en frontal.
- **Drupal Commerce** comme portail transactionnel de vente des formations et contenus pédagogiques.[cite:61][cite:63][cite:66]
- **Moodle** comme LMS principal et plateforme de consommation des contenus après attribution des droits d'accès.[cite:62][cite:65]
- **Keycloak** comme fournisseur d'identité et de SSO.
- **n8n** comme couche d'intégration visuelle no-code ou low-code entre Drupal Commerce et Moodle pour automatiser l'attribution des accès après achat simulé ou confirmé.[cite:73]

[cite:76][cite:77][cite:82]

- **Opencast** comme service vidéo compatible avec Moodle.[cite:12][cite:16][cite:70]
- **PostgreSQL** comme base de données unique lorsque cela est réaliste, avec des bases logiques séparées par service si applicable.
- **Redis** pour le cache ou les sessions lorsque pertinent.
- **Portainer CE** comme interface graphique de gestion et d'exploitation des conteneurs.[cite:33][cite:38]
- **Un volet minimal d'observabilité**, idéalement centré sur les journaux applicatifs, avec **Grafana + Loki** si votre équipe est capable de l'intégrer sans compromettre la qualité globale du projet.[cite:31][cite:53][cite:55]

Le projet n'a pas pour objectif de reproduire une architecture d'entreprise complète, mais plutôt de démontrer une compréhension claire et progressive des principes fondamentaux de sécurité appliqués à un environnement conteneurisé réaliste. Le parcours fonctionnel attendu est le suivant : un utilisateur consulte une offre de formation dans Drupal Commerce, réalise un paiement fictif, simulé ou en environnement sandbox, puis un workflow n8n déclenche l'attribution ou la synchronisation d'un droit d'accès dans Moodle, où le contenu est ensuite consommé.[cite:5][cite:60][cite:61][cite:73]

Résultats d'apprentissage visés

À la fin de ce travail, votre équipe devra être capable de :

- Concevoir une architecture multi-services cohérente et segmentée, intégrant une logique de commerce numérique, d'identité et de consommation pédagogique.
- Déployer une VM Linux à l'aide d'un mécanisme d'initialisation déclaratif avec cloud-init.
- Déployer une plateforme applicative conteneurisée avec Docker Compose.
- Mettre en place un reverse proxy, un WAF et des points d'entrée HTTPS.
- Configurer un mécanisme de SSO avec Keycloak, au minimum entre Keycloak et Moodle.
- Mettre en place une logique d'intégration visuelle avec n8n entre Drupal Commerce et Moodle, sans exiger de développement logiciel avancé.[cite:76][cite:77][cite:79]
- Appliquer des mesures de durcissement sur les images, les conteneurs et l'hôte.
- Réduire la surface d'attaque par une segmentation réseau et une exposition minimale des services.
- Intégrer une forme de visibilité opérationnelle et de journalisation utile en contexte d'incident.
- Documenter les choix architecturaux, les compromis, les limites et les risques résiduels.

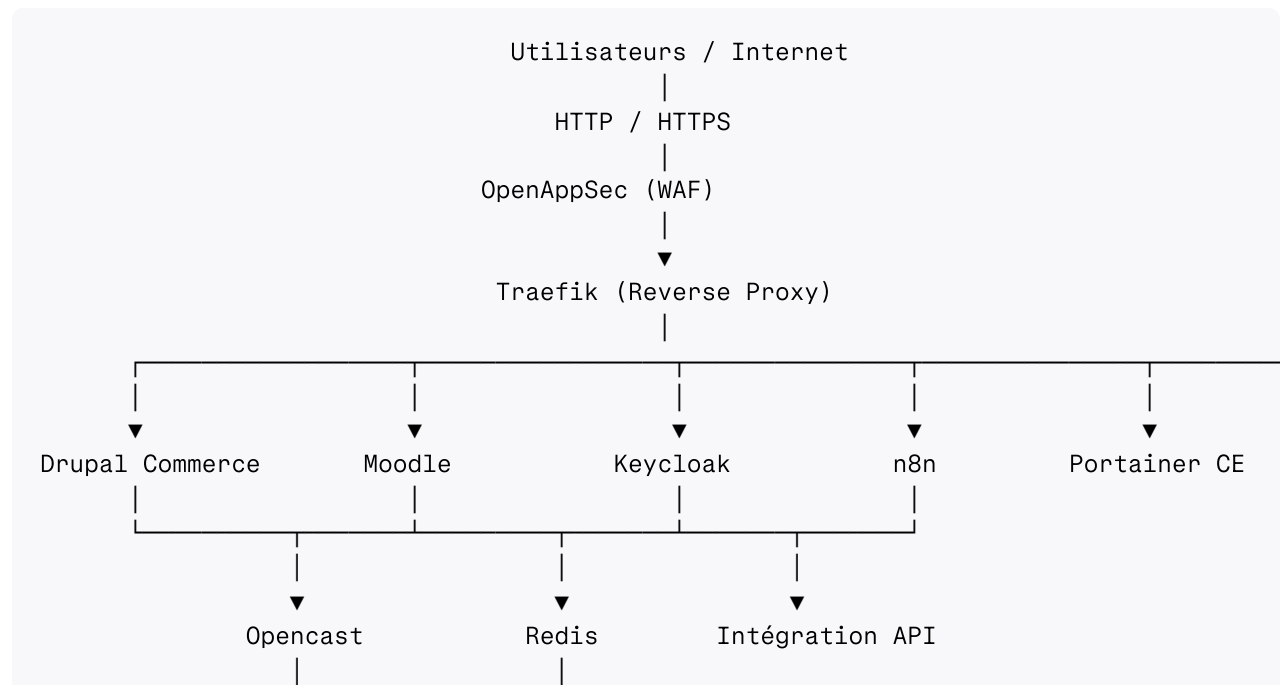
Contraintes obligatoires

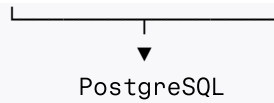
Les contraintes suivantes s'appliquent à tous les projets :

1. **VM as Code obligatoire.** Votre environnement doit être initialisé au moyen d'un fichier cloud-init. Ce fichier doit permettre de configurer au minimum l'utilisateur d'administration, les accès de base, les mises à jour initiales, l'installation de Docker et la préparation de l'environnement hôte.
2. **Docker Compose obligatoire.** Le déploiement applicatif doit être réalisé à l'aide de Docker Compose. Kubernetes n'est pas requis dans le cadre de ce travail.
3. **Une seule VM.** Le projet doit être réalisable sur une seule machine virtuelle Linux afin de limiter la complexité d'exploitation.
4. **Équipe de 3 à 5 personnes.** Les contributions individuelles doivent être identifiables.
5. **Base de données simplifiée.** Une seule instance PostgreSQL doit être privilégiée lorsque cela est techniquement réaliste pour les services choisis.
6. **Approche sécurité explicite.** Les mécanismes de sécurité doivent être démontrés et expliqués, et non seulement déclarés dans la documentation.
7. **Aucune programmation avancée obligatoire.** Les intégrations orientées produit doivent privilégier les outils graphiques, les connecteurs, les webhooks et les interfaces visuelles.
8. **Paiement simplifié autorisé.** Le paiement peut être fictif, simulé ou en environnement sandbox. Aucune intégration comptable complète n'est requise. Aucune preuve de paiement réelle n'est exigée.
9. **Livrables numériques obligatoires.** La remise doit comprendre une vidéo, un GitBook, un dépôt GitHub et un document de synthèse Word ou PowerPoint.

Architecture cible minimale

Votre architecture devra suivre un modèle semblable au suivant :





Parcours métier : vente sur Drupal Commerce → workflow n8n → accès Moodle
* Grafana/Loki recommandé lorsque l'équipe peut l'intégrer correctement

Ce schéma n'est pas figé. Vous pouvez l'ajuster si vous justifiez clairement vos choix. Toutefois, les principes suivants doivent être respectés :

- les services de données ne doivent pas être exposés directement à Internet;
- les points d'entrée doivent être limités et centralisés;
- les réseaux internes doivent être segmentés;
- les flux d'authentification doivent être documentés;
- le flux d'attribution d'accès après commande doit être documenté;
- l'architecture doit illustrer une logique de défense en profondeur.[cite:10][cite:24][cite:51]

Exigences techniques minimales

Votre solution devra démontrer les éléments suivants.

1. Couche VM as Code

Le fichier cloud-init doit permettre, au minimum :

- la création d'un utilisateur d'administration;
- la configuration de l'accès SSH;
- l'application des mises à jour initiales du système;
- l'installation de Docker Engine et de Docker Compose;
- la préparation des répertoires de travail;
- l'activation ou la configuration minimale d'un pare-feu hôte;
- la préparation de l'environnement de journalisation de base si applicable.

Vous devez expliquer ce que cloud-init configure automatiquement, ainsi que les limites de votre automatisation.

2. Couche conteneurisée

Votre déploiement Docker Compose doit inclure :

- les services applicatifs imposés;
- la présence d'une logique d'intégration claire entre Drupal Commerce, n8n et Moodle;
- les réseaux Docker nécessaires;
- les volumes persistants pertinents;
- la configuration minimale des variables d'environnement;

- des ports exposés limités au strict nécessaire;
- une logique claire de dépendances entre services.

3. Sécurité des images

Vous devez démontrer :

- l'utilisation d'images maintenues et identifiées par une version explicite;
- l'absence de dépendance inutile lorsque possible;
- au moins un mécanisme de validation ou de scan de sécurité d'image;
- une réflexion sur la provenance, la taille ou le niveau de confiance des images retenues.
[cite:14][cite:51]

4. Sécurité à l'exécution

Vous devez démontrer, lorsque possible :

- l'exécution de services avec un utilisateur non-root;
- l'absence de mode privilégié non justifié;
- la réduction des privilèges;
- l'usage de `security_opt` appropriés, par exemple `no-new-privileges:true` lorsque compatible;
- des limites de ressources;
- des volumes et systèmes de fichiers montés de manière prudente;
- des vérifications de santé pour les services critiques.[cite:47][cite:50][cite:52][cite:57]

5. Segmentation réseau

Votre architecture devra montrer une séparation logique entre :

- le réseau d'exposition;
- le réseau applicatif;
- le réseau de données.

Vous devrez être capables d'expliquer quels services communiquent entre eux, pourquoi ils en ont besoin, et quels accès sont volontairement interdits.

6. Intégration fonctionnelle sans codage avancé

Votre solution doit démontrer un flux métier minimal cohérent :

- un catalogue ou produit de formation dans Drupal Commerce;
- un paiement fictif, simulé ou sandbox;
- un déclenchement de workflow dans n8n;

- une attribution, inscription ou activation d'accès côté Moodle;
- une consommation du contenu d'apprentissage dans Moodle.

Aucune intégration comptable complète n'est requise. Aucune preuve de paiement réelle n'est exigée. L'évaluation portera sur la cohérence fonctionnelle, l'intégration visuelle et la sécurité de l'architecture, et non sur le développement applicatif.[cite:73][cite:76][cite:81]

7. Défense en profondeur

Votre plateforme doit montrer plusieurs couches de protection complémentaires, par exemple :

- WAF en frontal;
- reverse proxy centralisé;
- segmentation réseau;
- IAM et SSO;
- réduction des privilèges;
- logs et traçabilité;
- limitation de l'exposition externe.

8. Identité et SSO

Vous devez configurer **Keycloak** comme fournisseur d'identité. Le minimum attendu est un SSO fonctionnel entre Keycloak et Moodle. L'intégration SSO avec Drupal Commerce est souhaitable, mais elle peut être traitée comme une bonification ou comme une amélioration future si votre équipe préfère stabiliser d'abord le noyau de la plateforme.

9. Observabilité et exploitation

Portainer CE est obligatoire afin d'offrir une vue graphique pédagogique de l'environnement conteneurisé. Vous devrez l'utiliser pour illustrer les conteneurs, les réseaux, les volumes, les états et certains journaux d'exécution.[cite:33][cite:38]

Un mécanisme additionnel de journalisation centralisée est fortement recommandé. **Grafana Loki** constitue une option légère et pédagogique pour agréger, consulter et rechercher des journaux issus de plusieurs services.[cite:31][cite:53][cite:55]

Orientations pédagogiques

L'objectif n'est pas de construire la plateforme la plus complexe possible. Une plateforme partiellement plus simple, mais bien comprise, bien sécurisée et bien documentée, sera mieux évaluée qu'une architecture trop ambitieuse et instable.

Vous êtes encouragés à :

- justifier vos choix plutôt que d'empiler des outils;
- privilégier la configuration, l'intégration visuelle et les outils graphiques lorsque cela suffit;

- expliquer les limites des images ou produits choisis;
- démontrer au moins un scénario simple d'investigation ou de diagnostic;
- expliciter vos hypothèses de sécurité;
- montrer ce qui a été automatisé et ce qui reste manuel.

Vous n'êtes pas pénalisés pour ne pas atteindre un niveau de production industriel complet. En revanche, vous serez pénalisés si vous exposez inutilement des services sensibles, si vous laissez des secrets dans le dépôt Git, ou si vous êtes incapables d'expliquer le fonctionnement de votre propre architecture.[cite:10][cite:56]

Livrables obligatoires

La remise finale comprend **quatre livrables obligatoires**.

1. Vidéo de démonstration

- Durée obligatoire : 15 min 00 s à 15 min 59 s maximum.
- Hébergement obligatoire sur Microsoft 365 de l'université, par exemple Microsoft Stream ou un service M365 équivalent.
- La vidéo doit être accessible à l'enseignant et, si requis, à l'équipe de correction.

La vidéo doit au minimum contenir :

- une brève introduction de l'équipe;
- la présentation du mandat et du périmètre;
- la démonstration de la VM as Code via cloud-init;
- la présentation du schéma d'architecture;
- la démonstration fonctionnelle de la plateforme, y compris le parcours de vente et d'attribution d'accès;
- la présentation des mécanismes de sécurité;
- la démonstration minimale du SSO;
- la démonstration d'un volet d'observabilité ou de diagnostic;
- les limites, difficultés et améliorations futures.

2. GitBook

Le GitBook doit contenir une documentation structurée et lisible comprenant au minimum :

- la présentation du projet;
- l'architecture;
- la description de la VM et du fichier cloud-init;
- le déploiement Docker Compose;
- la segmentation réseau;

- les mesures de sécurité;
- le SSO;
- le workflow n8n et le flux d'attribution des accès après commande;
- l'observabilité;
- les limites et les risques résiduels;
- la contribution des membres de l'équipe.

3. Dépôt GitHub

Le dépôt GitHub doit contenir l'ensemble du code source et de la documentation technique utile. Il doit inclure, au minimum :

- le ou les fichiers cloud-init;
- les fichiers Docker Compose;
- les configurations associées aux services;
- un README.md clair;
- un fichier .env.example sans secrets réels;
- les scripts utiles au démarrage ou à la validation;
- des captures ou traces utiles si nécessaire.

Une structure de dépôt claire sera valorisée.

4. Document de synthèse

Vous devez remettre au choix :

- un document Word de 10 pages maximum, page de garde incluse; ou
- un document PowerPoint de 10 pages maximum, page de garde incluse.

Ce document doit être une synthèse décisionnelle, et non une duplication complète du GitBook. Il doit permettre à un évaluateur de comprendre rapidement votre architecture, vos choix, vos protections, vos limites et votre répartition du travail.

Structure suggérée du dépôt GitHub

La structure suivante est recommandée :

```
repo/
├─ cloud-init/
│   └─ user-data.yaml
│   └─ README.md
├─ compose/
│   └─ docker-compose.yml
│   └─ .env.example
│   └─ traefik/
│   └─ openappsec/
│   └─ drupal-commerce/
```

```

|   ├── moodle/
|   ├── keycloak/
|   ├── n8n/
|   ├── opencast/
|   ├── portainer/
|   └── observability/
├── docs/
|   ├── architecture/
|   ├── security/
|   ├── operations/
|   └── testing/
├── media/
└── README.md

```

Vous pouvez proposer une autre structure si elle demeure cohérente et facile à évaluer.

Contenu suggéré de la vidéo

La structure suivante est fortement recommandée :

1. Présentation de l'équipe et du mandat.
2. Présentation de la VM as Code et du fichier cloud-init.
3. Présentation de l'architecture logique et du schéma réseau.
4. Démonstration du déploiement ou de l'environnement opérationnel.
5. Démonstration de Drupal Commerce, du flux n8n, de Moodle et du SSO Keycloak.
6. Présentation du reverse proxy, du WAF et des protections choisies.
7. Démonstration de Portainer CE et, si disponible, de la journalisation centralisée.
8. Conclusion courte : limites, risques résiduels, apprentissages.

Grille d'évaluation — 40 points

Critère	Attentes	Points
VM as Code	Fichier cloud-init fonctionnel, clair, utile et reproductible	5
Architecture et schéma	Diagramme lisible, cohérent, justifié, avec flux, zones de confiance et parcours métier	5
Déploiement conteneurisé	Stack Compose fonctionnelle, cohérente et bien structurée	4
Sécurité des images	Versions explicites, choix justifiés, scan ou validation minimale	4
Sécurité à l'exécution	Non-root, réduction des privilèges, options de sécurité, ressources	5
Segmentation réseau	Réseaux séparés, exposition minimale, flux justifiés	4
Intégration fonctionnelle	Drupal Commerce, n8n et Moodle intégrés selon un flux crédible sans codage avancé	4
IAM et SSO	Keycloak configuré, SSO Moodle fonctionnel, gestion minimale des rôles	4
Reverse proxy et WAF	Traefik et OpenAppSec intégrés et expliqués	2
Observabilité	Portainer CE opérationnel, plus logs ou diagnostic crédible	2

Critère	Attentes	Points
Qualité des livrables	Vidéo conforme, GitBook clair, GitHub propre, Word/PPT synthétique	5
Total		40

Attentes par critère

VM as Code

Le fichier cloud-init doit faire plus qu'installer quelques paquets. Il doit réellement illustrer une approche déclarative d'initialisation de la VM et éviter un maximum d'étapes manuelles répétitives.

Architecture et schéma

Le schéma doit être lisible, légendé et cohérent avec ce qui est réellement déployé. Un schéma très beau mais non conforme au déploiement réel sera pénalisé.

Déploiement conteneurisé

La stack doit être stable, compréhensible et assez mature pour être démontrée. La reproductibilité comptera davantage que la sophistication.

Sécurité des images et à l'exécution

Les protections doivent être visibles dans le code, dans la configuration ou dans la démonstration. Il ne suffit pas d'écrire que le projet est "sécurisé".

Intégration fonctionnelle

Le scénario attendu doit démontrer un enchaînement réaliste entre la commande de formation dans Drupal Commerce, le workflow n8n et l'attribution d'un accès exploitable dans Moodle. La solution peut être partiellement simulée, mais elle doit être cohérente et démontrable.

Observabilité

Portainer CE doit servir à visualiser l'environnement. Si vous ajoutez Loki, Grafana ou un autre mécanisme léger de journalisation, vous devez montrer en quoi cela vous aide à comprendre un comportement normal ou anormal, y compris dans le flux Drupal Commerce → n8n → Moodle.

Pénalités majeures

Les situations suivantes entraîneront une pénalité importante, voire l'échec de certains critères :

- secrets réels committés dans GitHub;
- conteneurs applicatifs exécutés en mode privilégié sans justification sérieuse;

- exposition directe de PostgreSQL, Redis, n8n ou d'interfaces d'administration sensibles sur Internet;
- incapacité d'expliquer les réseaux, les volumes, les flux ou les dépendances;
- absence réelle de cloud-init malgré une revendication de VM as Code;
- vidéo dépassant 15 min 59 s;
- dépôt incomplet ou non exploitable.

Conseils de réalisation

Pour réussir ce travail, il est conseillé de procéder par étapes :

1. initialiser la VM avec cloud-init;
2. valider Docker et l'environnement hôte;
3. démarrer une version minimale de la stack;
4. ajouter progressivement Traefik, Drupal Commerce, Keycloak, n8n et Moodle;
5. intégrer ensuite les protections et l'observabilité;
6. documenter au fur et à mesure;
7. préparer la vidéo comme une démonstration argumentée et non comme une simple visite d'écrans.

Un projet simple, stable et bien expliqué sera mieux noté qu'un projet trop ambitieux, incomplet ou peu maîtrisé.

Remise

Les modalités exactes de remise seront précisées par l'enseignant. Sauf indication contraire, la remise devra comprendre :

- le lien de la vidéo hébergée sur Microsoft 365;
- le lien du GitBook;
- le lien du dépôt GitHub;
- le document Word ou PowerPoint;
- tout accès additionnel requis pour la consultation.

Assurez-vous que les liens sont fonctionnels, que les permissions d'accès sont adéquates et que tous les livrables sont cohérents entre eux.

Rappel important

Le but de ce travail n'est pas uniquement de faire fonctionner des conteneurs. Il s'agit de démontrer votre capacité à **penser l'architecture**, à **automatiser le socle**, à **réduire les risques**, à **justifier vos choix** et à **communiquer clairement un système conteneurisé sécurisé**.